# Docker User Manual

NECKER

December 16, 2025

## Contents

# 1 Architecture and Fundamental Concepts

### Virtualization: VM vs Container

The substantial difference lies in the *level of abstraction*.

- **Virtual Machines (Hardware Virtualization):** A VM emulates an entire physical machine. A complete **Guest OS** (with its own Kernel, drivers, memory management) runs on top of the Hypervisor.
  - *Pros:* Total isolation.
  - *Cons:* Heavy (GBs of space), slow startup (OS boot), waste of resources (Kernel duplication).
- **Containers (OS Virtualization):** A container virtualizes the Operating System, not the hardware. All containers share the same **Host Kernel**. Docker Engine isolates processes (using Namespaces) and resources (using Cgroups), but it does not emulate hardware.
  - *Pros:* Extremely lightweight (MBs), instantaneous startup (it is just a process), high density.
  - *Cons:* Limited to the architecture of the host operating system.

### The OOP Paradigm: Images and Containers

For a developer, the best way to understand Docker is through the analogy with Object-Oriented Programming (Classes and Objects).

## 1.1 The Image (The Class)

In programming, a **Class** is a static model, a "blueprint" that defines properties and methods, but does not "live" in memory by itself until it is instantiated.

In the Docker world, the **Image** is the equivalent of the Class (or rather, the compiled/binary class):

- **Immutable:** Once built (‘docker build‘), it cannot be modified. Just like you do not modify the bytecode of a class at runtime.
- **Static:** It is a read-only package that contains everything needed for execution: source code, runtime (e.g., JVM, Node), system libraries, and dependencies.
- **Layered:** It is built in layers. Every instruction in the Dockerfile creates a new immutable layer.

## 1.2 The Container (The Object)

In programming, an **Object** is an *instance* of a class. It occupies memory, has a state, and a lifecycle. You can create infinite objects from the same class.

In the Docker world, the **Container** is the equivalent of the Object:

- **Runtime:** It is the image coming to life. When you run ‘docker run‘, you are performing an *instantiation* operation (‘new Image()‘).
- **Mutable (with ephemerality):** Unlike the image, the container can be modified. Docker adds a thin *Writable Layer* on top of the image layers. If the container writes a file, it writes it here.
- **Isolated:** If you create two containers from the same image (e.g., two Postgres databases), they are two separate instances. If you modify data in one, the other is not affected (just like modifying ‘obj1.x‘ does not change ‘obj2.x‘).

Summary for Programmers

**Dockerfile** ≈ Source Code (.java / .cpp)
**Docker Image** ≈ Compiled Class / Static executable
**Docker Container** ≈ Object in Heap / Running process

# 2 Deep Dive: Docker Internals and Linux Kernel

Docker is not a monolithic technology, but a wrapper (previously via LXC, now via `libcontainer/runc`) that orchestrates three fundamental primitives of the Linux Kernel. Understanding these concepts is the key to advanced debugging and performance optimization.

## 2.1 Namespaces: The Illusion of Isolation

Namespaces are a kernel feature that partitions system resources so that a set of processes sees only a portion of the resources. When Docker starts a container, it invokes the `clone()` syscall passing specific flags (e.g., `CLONE_NEWPID`, `CLONE_NEWNET`).

- **PID Namespace (Process ID):** Isolates the process tree.
  - *Inside:* The main process of the container sees itself as **PID 1**. This is crucial because PID 1 has special responsibilities (signal handling, reaping orphan/zombie processes).
  - *Outside:* On the host, that same process has a regular PID (e.g., 4532).
- **NET Namespace (Networking):** Provides the container with a complete and private network stack: its own interfaces (e.g., `eth0`), routing tables, iptables rules, and sockets. Docker uses *veth pairs* (virtual ethernet cables) to connect the container's namespace to the host's bridge (`docker0`).
- **MNT Namespace (Mount Points):** Allows the container to have a different view of the filesystem. A process can mount/unmount directories without affecting the host. This is what allows the container to see its own `/` root separate from the host's root.
- **USER Namespace (Advanced Security):** Allows the mapping of user IDs. The process can be `root` (UID 0) inside the container but be mapped to an unprivileged user (e.g., UID 1000) on the host. This drastically mitigates the risks of *Container Breakout*.

## 2.2 Cgroups (Control Groups)

While Namespaces manage what the container can *see*, Cgroups manage what the container can *use*. They are directory hierarchies located in `/sys/fs/cgroup`.

- **Resource Limiting:** You can set a hard limit on memory (e.g., 512MB). If the container exceeds the limit, the Kernel invokes the **OOM Killer** (Out of Memory Killer) and terminates the process.
- **Prioritization:** You can assign weights to the CPU (CPU Shares). If the system is idle, a container can use 100% of the CPU. If there is contention, Cgroups guarantee resources based on the assigned weights.
- **Accounting:** Cgroups track how much resource each container consumes. This is the foundation on which commands like `docker stats` work.

### 2.3 Union File System & Copy-on-Write (CoW)

This is the magic that makes containers fast to start and lightweight on disk. Docker uses storage drivers (usually **Overlay2**) that implement a Union File System.

Imagine the image as a stack of transparent sheets (Layers).

1. **LowerDir (Image Layers):** These are the layers of the downloaded image (e.g., base Ubuntu + Python + App Code). They are **Read-Only** and can never be modified.
2. **UpperDir (Container Layer):** When you start a container, Docker adds an empty layer on top. This is the only **Read-Write** layer.
3. **Merged View:** The container sees the combination of these layers as a single, coherent filesystem.

#### 2.3.1 The Copy-on-Write (CoW) Mechanism

What happens when the container wants to modify a file that exists in a read-only image layer?

- **Read:** If the container reads a file, it reads it directly from the lower layer (native speed).
- **Write:** If the container wants to modify `/etc/config.ini` located in the image, the OverlayFS driver first **copies** the file from the lower layer (LowerDir) to the upper writable layer (UpperDir). The modification takes place on the copy.
- **Delete:** If you "delete" a file from the image, Docker actually creates a *Whiteout File* in the upper layer that hides the underlying file without actually removing it.

**Performance Tip for Programmers**

The CoW strategy has an I/O cost.
**Best Practice:** Do not use the container's filesystem for write-intensive workloads (e.g., Databases), as data will be deleted when the container is removed. Use **Volumes** (host directories connected to the container), which bypass the OverlayFS driver and write directly to the host disk at native speed.

## 3 Advanced Build Strategies

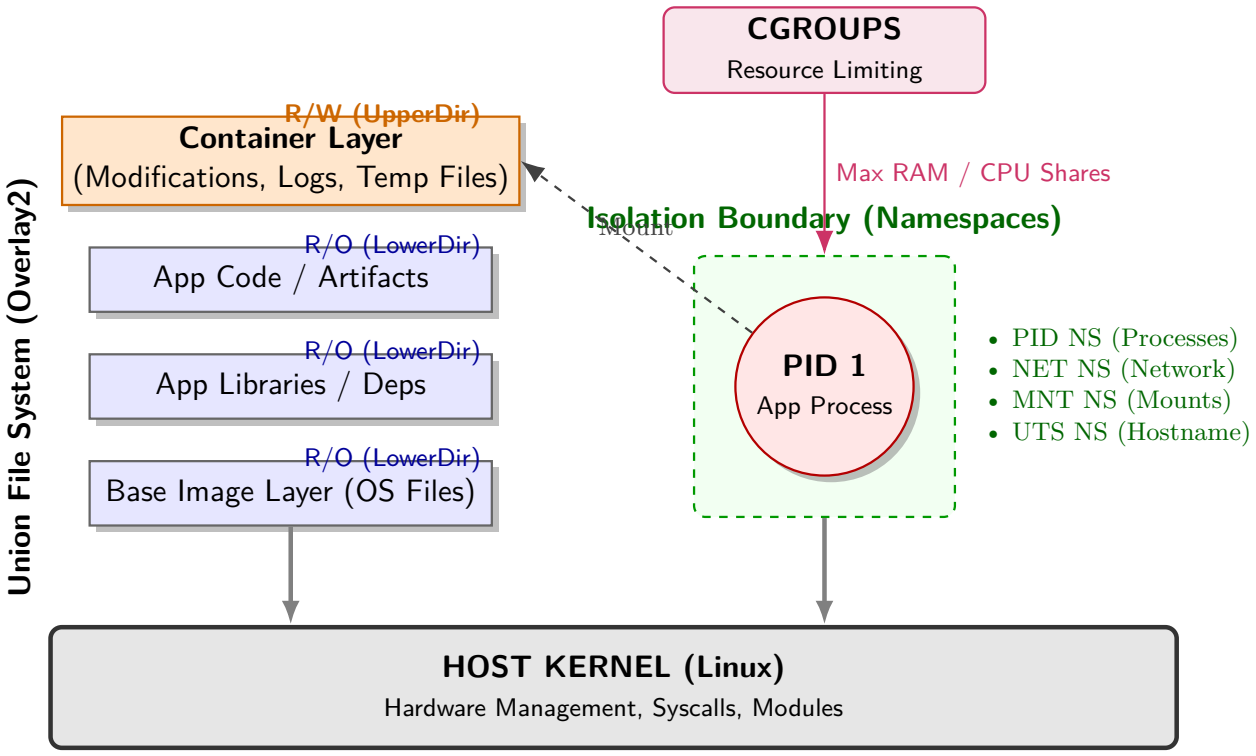
The Dockerfile is not just an installation script; it is the immutable definition of the production environment. Writing it poorly leads to slow builds, giant images, and security vulnerabilities.

### 3.1 Optimizing Layer Cache

Every instruction in the Dockerfile ('RUN', 'COPY', 'ADD') generates a new layer. Docker utilizes an aggressive caching system: if the content of a layer hasn't changed compared to the previous build, Docker reuses the cached layer.

**The key concept is the instruction order:** you must place instructions that change rarely (e.g., system dependency installation) at the top, and those that change often (e.g., source code) at the bottom.

```
1  # Defines the starting image (Debian-based Linux OS + Python 3.9 interpreter).
2  FROM python:3.9-slim
3
4  # Sets global environment variables for the container:
5  # - PYTHONDONTWRITEBYTECODE: Prevents creating .pyc files (useless in stateless
   containers).
```




```
# - PYTHONUNBUFFERED: Sends logs directly to stdout without buffering (essential for Docker).
ENV PYTHONDONTWRITEBYTECODE=1 \
PYTHONUNBUFFERED=1

# Creates the /app directory (if it doesn't exist) and sets the working directory pointer there (equivalent to mkdir + cd).
WORKDIR /app

# Copies the requirements file from the host to the container filesystem.
# Executed separately to leverage the Layer Cache if dependencies do not change.
COPY requirements.txt .

# Executes the library installation inside the container during the build.
# --no-cache-dir avoids saving the pip cache, reducing the image size.
RUN pip install --no-cache-dir -r requirements.txt

# Copies all the remaining source code from the current host folder to the container's WORKDIR.
COPY . .

# Creates a new system user named 'myuser' to avoid running the app as Root.
RUN useradd -m myuser

# Switches the active user. All subsequent instructions (and the app) will run as 'myuser'.
USER myuser

# Declares (for documentation purposes) that the service listens on TCP port 5000.
EXPOSE 5000

# Defines the command launched when the container starts (PID 1).
# We use the JSON array syntax (Exec form) to correctly handle stop signals.
CMD ["python", "app.py"]

```

### 3.2 Exec Form vs Shell Form (Signal Handling)

This is a detail that 90% of tutorials ignore, but it is vital for production stability. There are two ways to write ‘CMD‘ or ‘ENTRYPOINT‘.

- **Shell Form:** `CMD python app.py`
  - Docker starts the command by wrapping it in `/bin/sh -c`.
  - **The Problem:** The shell becomes the PID 1 process, and your app becomes a child process. Shells often **do not forward signals** (like `SIGTERM` or `SIGINT`) to their children.
  - *Result:* When you execute `docker stop`, the container does not stop gracefully; instead, it waits 10 seconds and is brutally killed (`SIGKILL`), potentially corrupting data.
- **Exec Form (JSON Array):** `CMD ["python", "app.py"]`
  - Docker starts the executable directly.
  - Your app is **PID 1**. It immediately receives stop signals and can perform a clean shutdown (closing DB connections, finishing ongoing requests).
  - **Best Practice:** ALWAYS use the square bracket notation.

### 3.3 CMD vs ENTRYPOINT: The Interaction Matrix

Often confused, they actually work together.

- **ENTRYPOINT:** Defines the "fixed" executable of the container.
- **CMD:** Defines the default arguments passed to the ENTRYPOINT.

**Common Pattern: The Configurable Executable**

If you want a container to behave like a binary command:

```
ENTRYPOINT ["/bin/ping"]
CMD ["localhost"]

```

- `docker run myimg` → Executes `ping localhost`
- `docker run myimg google.com` → Executes `ping google.com` (CMD overridden)

### 3.4 Multi-Stage Builds: Separating Build and Runtime

Introduced to solve the problem of huge images in compiled languages (Go, Java, C++, Rust) or modern frontends (React/Angular).

The historical problem was: "I need `gcc`, `maven`, or `node_modules` to compile, but I do not want them in production because they add weight and vulnerabilities".

With Multi-Stage builds, you use a "fat" image to compile and copy only the final artifact into a "slim" image.

```
# --- STAGE 1: The Construction Site (Builder) ---
# We use a full image with the Go compiler
FROM golang:1.18 AS builder
WORKDIR /app
COPY . .
# We compile a static binary (without dynamic dependencies)
RUN CGO_ENABLED=0 GOOS=linux go build -o myapp .

# --- STAGE 2: The Finished Product (Runtime) ---
# We use Alpine (5MB) or even Scratch (0MB - an empty image)
FROM alpine:latest
WORKDIR /root/
# We copy ONLY the binary file from the previous stage
COPY --from=builder /app/myapp .

# Result: A 10MB image instead of 800MB
CMD ["./myapp"]

```

Listing 1: Go Multi-Stage Example

### 3.5 The .dockerignore File

Often forgotten, it works exactly like `.gitignore`. Before starting the build, the Docker client sends the entire content of the current folder (Build Context) to the Docker daemon.

If you don't have a `.dockerignore`, you might inadvertently send:

- The `.git` folder (huge and useless).
- Local folders like `venv` or `node_modules` (they break the Linux build if you develop on Mac/Windows).
- Files containing secrets (`.env`).

**Example .dockerignore:**

```
.git
.env
venv
__pycache__
node_modules
Dockerfile
.dockerignore
```

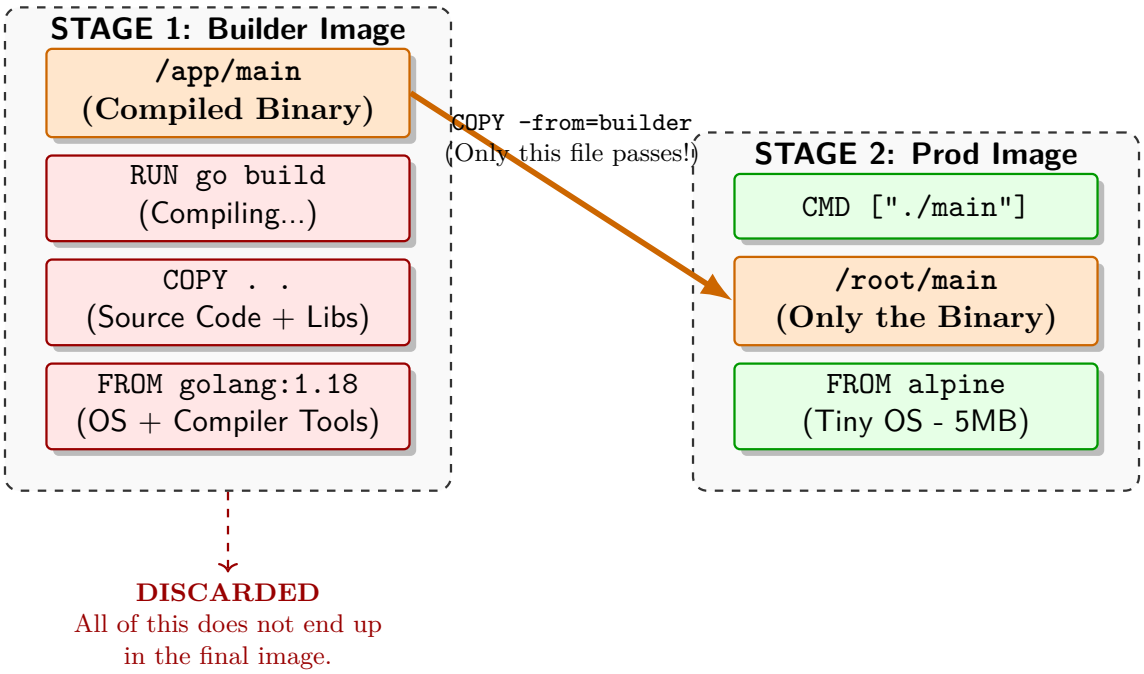



Figure 1: Multi-Stage Build visualization. Note how the entire compilation environment (Stage 1) is discarded, transferring only the final executable into the production environment (Stage 2).

# 4 Networking: How Containers Communicate

Docker uses standard Linux networking primitives to route packet traffic.

### 1. Virtual Hardware: Bridge and VETH Pairs

When Docker starts, it creates a software virtual switch on the host called `docker0`.
But how does the container connect to this switch?
Via **VETH Pairs (Virtual Ethernet)**.

Imagine an Ethernet cable with two ends:

1. **Container Side (`eth0`):** Resides inside the container's Network Namespace. To the process, this looks like a real physical network card.
2. **Host Side (`vethXYZ`):** The other end of the cable remains in the Host Namespace and is bridged ("plugged") into `docker0`.

*Result:* All containers connected to the bridge can exchange packets as if they were computers connected to the same physical switch.

## 4.1 Network Modes Comparison

**A. BRIDGE (Default)**
The container is isolated. It has its own private IP (e.g., `172.17.0.2`) and cannot be reached from the outside.

- **Pros:** Complete isolation, no port collisions.
- **Cons:** Requires NAT to go out and Port Mapping to come in.

**B. HOST (`-network host`)**
Removes network isolation. The container "borrows" the host's network card.

- **Pros:** Native performance (zero NAT overhead). Ideal for VoIP or streaming.

- **Cons:** If the container listens on port 80, it occupies the real port 80 of the machine. You cannot run two identical containers.

### 4.2 External Communication: Port Mapping (DNAT)

How does `-p 8080:80` actually work? Docker does not act as a user-space proxy; instead, it manipulates the Kernel via **IPTables**.

Docker inserts a rule into the Linux firewall's `PREROUTING` chain:

```
# Pseudo-IPTables rule created by Docker
-A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 172.17.0.2:80

```

**Translation:** "Any TCP packet arriving at the Host's port 8080, change its destination address to the Container's IP at port 80". This is **Destination NAT**.

### 4.3 Internal Communication: DNS and Service Discovery

A common mistake is using container IPs (which change upon restart). Docker resolves this problem with an internal DNS server.

- **The DNS Server (`127.0.0.11`):** Docker injects this nameserver into the `/etc/resolv.conf` file of every container.
- **Service Discovery:** In Docker Compose, a custom network is created automatically. The DNS automatically maps the **service name** (e.g., `db`, `backend`) to its current IP.

**Watch out for Localhost!**

Inside a container, `localhost` (127.0.0.1) is the container itself!
If your Node.js backend tries to connect to Mongo using `localhost:27017`, it will fail because it looks for Mongo *inside* the Node container (connecting to itself in the wrong place).
**Solution:** Use the service name instead (e.g., `mongodb:27017`).

# 5 Persistence and Storage Strategy

The container's filesystem is designed to be ephemeral. Understanding *where* data goes is the difference between a robust app and a catastrophic data loss.

## 5.1 Volumes (Production / Databases)

Volumes are first-class objects managed by Docker. They reside in a part of the host filesystem managed EXCLUSIVELY by Docker (on Linux: `/var/lib/docker/volumes/`).

- **Advantages:**
  - Independent from the host directory structure.
  - Native I/O performance (especially on Docker Desktop for Mac/Windows, where bind mounts are slow).
  - Easy to back up and migrate (`docker volume create`, `docker volume ls`).
- **Best Use Case:** Databases (Postgres, MySQL), persistent key storage, production logs.

## 5.2 Bind Mounts (Development / Configuration)

They map an exact file or folder from the host into the container.
Example: `-v /home/user/project:/app`

- **Advantages:**
  - **Hot Reloading:** You modify the source code in your editor on the host, and the container sees the change immediately. Essential for web development (Node, Python, PHP).
  - Quick sharing of static configuration files (e.g., passing an `nginx.conf` on the fly).
- **The Permissions Problem (Permission Hell):** If the process in the container runs as `root` (default) and creates a file inside a bind mount, that file will appear on the host owned by `root`. You (as a regular user) won't be able to delete or modify it without `sudo`.
  *Solution:* Use the `user: "${UID}:${GID}"` flag in the compose file to align users.

## 5.3 tmpfs Mounts (Security)

Data is written only to the host's RAM. It never touches the disk.

- **Use Case:** Storing secrets, temporary tokens, or data that absolutely must not persist if the container turns off.

**Storage Decision Matrix**

- Source Code (Dev) → **Bind Mount**
- Database / Persistent Data → **Volume**
- Static Configuration Files → **Bind Mount** or **ConfigMap** (in K8s)
- Tokens / Secrets → **tmpfs**

# 6 Docker Compose: Orchestrazione e Infrastructure as Code

While `docker run` is great for starting "disposable" containers, it is unsuited for managing complex architectures. **Docker Compose** is the tool that allows you to define and manage multi-container applications.

## 6.1 The Paradigm Shift: Imperative vs Declarative

- **Docker Run (Imperative):** You have to tell the system *what to do* step by step. *"Download this, then start that, then connect the network..."*. It is manual, error-prone, and hard to share with the team.
- **Docker Compose (Declarative):** You describe in a file (YAML) *how the final state* of the system should look. *"I want a web service connected to a postgres database"*. Docker takes care of creating the resources needed to reach that state.

### Advantages over Docker Run

1. **Infrastructure as Code (IaC):** The entire infrastructure is defined in a file (`docker-compose.yml`) which can be versioned with Git. A new developer only needs to run `git clone` and `docker compose up` to have the environment ready.
2. **Automatic Networking:** Compose automatically creates an internal network dedicated to the project. Services "see" each other using their names (Service Discovery), without having to manage IPs.
3. **Configuration Persistence:** You don't have to remember kilometer-long strings of `-v`, `-p`, `-e` flags. It is all written in the file.

## 6.2 Anatomy of a docker-compose.yml

The YAML file is indentation-sensitive. Here is a realistic example of a Full-Stack stack (Python + Postgres).

```yaml
version: '3.8'  # Specification version

services:       # Definition of containers (Services)

# --- SERVICE 1: THE WEB BACKEND ---
web:
build: .            # Build the image from the current Dockerfile
container_name: my_python_app
ports:
- "5000:5000"     # Exposure: Host:Container
volumes:
- .:/app          # Bind Mount: Live development (Host code -> Container)
environment:        # Environment variables
- DB_HOST=db      # Note: 'db' is the service name below
- DB_PASS=secret
depends_on:         # Startup order
- db
networks:
- backend-net

# --- SERVICE 2: THE DATABASE ---
db:
image: postgres:13  # Uses a ready-made image from Docker Hub
restart: always     # Always restart if it crashes
environment:
POSTGRES_PASSWORD: secret
POSTGRES_DB: myappdb
volumes:
- pg-data:/var/lib/postgresql/data # Named Volume (Real persistence)
networks:
- backend-net

# Definition of Volumes (Persistent storage managed by Docker)
volumes:
pg-data:

# Definition of Networks
networks:
backend-net:
driver: bridge

```

## 6.3 Keywords Analysis

- **`services:`** The list of containers to launch. In the code above: `web` and `db`.
- **`build` vs `image`:**
  - `build: .` says "Create the image using the Dockerfile in this folder".
  - `image: postgres` says "Download the official ready-to-use image".
- **`depends_on`:** Controls the startup order. It says "Start `web` only after starting `db`". *Warning: Docker only waits for the DB container to be "running", not for the Postgres process to actually be ready to accept connections. An application retry-logic or a "wait-for-it" script is often needed in the application code.*
- **`networks`:** Defines the security perimeter. Containers in different networks cannot talk to each other.

### 6.4 The Operational Workflow

How is it used in daily life?

- **Startup:** `docker compose up -d`
  Reads the file, creates the network, the volumes, and starts the containers in the background (Detached).
- **Stop and Cleanup:** `docker compose down`
  Stops the containers and **removes** the virtual network. *Note: By default, it does NOT delete volumes (DB data is safe). To delete data as well, use* `-volumes`.
- **Logs:** `docker compose logs -f`
  Shows an aggregated real-time log output from all services (useful for cross-debugging).
- **Rebuild:** `docker compose up -d -build`
  Forces the image recompilation if you changed the Dockerfile or dependencies.

## 7 Commands Cheat-Sheet

- `docker build -t name:tag .` : Builds the image.
- `docker run -d -p 80:80 -name web nginx` : Starts a container in the background, mapping host port 80 to container port 80.
- `docker ps -a` : Lists containers (including stopped ones).
- `docker exec -it <id> bash` : Opens a shell inside an active container.
- `docker logs -f <id>` : Follows logs in real time.
- `docker system prune -a` : Cleans everything up (unused images, stopped containers).
- `docker compose up -d -build` : Starts the app, compiling it again if necessary.